

May 2025 update- AI chatbots have advanced to the point where it probably makes sense to ask an AI to retrieve the average monthly temperature, humidity, precipitation, sunlight, etc. data from a Wikipedia-style climate box

Composite Los Angeles (USC normals + LAX dew points)

https://docs.google.com/spreadsheets/d/1FeHQVID8msfRKVbp8GQHJ_AvZWDUWv-rHufEHExGZb4/edit#gid=1036500340

Coordinates: 34.0206°N 118.2848°W

```
Record high (°C),35,35,37,41,39,44,43,41,45,42,38,33,45
Mean maximum (°C),28.3,28.2,29.9,32.3,31.6,31.7,34.2,35.1,37.4,35.4,31.6,27.2,38.6
Mean daily maximum (°C),20,20,21.1,22.4,23.2,25.1,27.8,28.9,28.3,25.9,22.7,19.7,23.8
Daily mean (°C),14.7,15,16.2,17.6,18.8,20.7,22.9,23.7,23.1,20.7,17.2,14.3,18.8
Mean daily minimum (°C),9.4,10,11.3,12.7,14.5,16.3,18.2,18.6,17.9,15.5,11.7,9,13.8
Mean minimum (°C),5.2,6.1,7.4,9.4,11.9,14.1,16.2,16.5,15.1,12.1,7.4,4.7,4
Record low (°C),-2,-2,-1,2,4,8,9,9,7,4,1,1,-2
Average rainfall (mm),84,92,57,18,8.1,2.3,0.51,0,3.3,15,20,63,363.21
Average rainy days (≥ 0.01 in),6.1,6.3,5.1,2.8,1.9,0.5,0.4,0.1,0.4,2.2,2.8,5.5,34.1
Sunshine hours,225.3,222.5,267,303.5,276.2,275.8,364.1,349.5,278.5,255.1,217.3,219.4,"3,254.20"
Average dew point (°C),5.2,6.9,8.1,9.5,11.5,13.6,15.6,16.2,15.1,12.3,8.2,5.2,10.6
```

https://github.com/jambao24/MATLAB_climatedata_modeling/blob/main/TempSimul3%20original%20code.rtf

```
# NOTE: contents of original notebook streamlined and formatted using Deepseek
# CASE 1- relative humidity stats are daily, has sunshine hours not percentages
# CASE 2- relative humidity stats are daily, has sunshine percentages
# CASE 3- relative humidity stats are afternoon (3 pm), has sunshine hours not percentages
# CASE 4- relative humidity stats are afternoon (3 pm), has sunshine percentages

# =====
# IMPORTS
# =====
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import math
from scipy.interpolate import CubicSpline

# =====
# booleans for defining cases
# =====
hasDewpoint = False      # Sylhet data comes as RH, not dewpoint
humidityPM = False      # RH stats are daily averages, not afternoon (3 pm)
sunshinePerc = False     # Sylhet provides sunshine HOURS, not percentages

# =====
# MISSING HELPER (not defined in the original notebook)
# =====
def assign_monthly_avg_to_days(monthly_avgs):
    days = np.zeros(365)
    days[0:31] = monthly_avgs[0]
    days[31:59] = monthly_avgs[1]
    days[59:90] = monthly_avgs[2]
    days[90:120] = monthly_avgs[3]
    days[120:151] = monthly_avgs[4]
    days[151:181] = monthly_avgs[5]
    days[181:212] = monthly_avgs[6]
    days[212:243] = monthly_avgs[7]
    days[243:273] = monthly_avgs[8]
    days[273:304] = monthly_avgs[9]
    days[304:334] = monthly_avgs[10]
    days[334:365] = monthly_avgs[11]
    return days

# =====
# CORE FUNCTIONS (copied from the notebook, unchanged)
# =====
```

```

def calculate_mean_monthly_temps(annual_temps):
    monthly_temps = np.zeros(12)
    monthly_temps[0] = np.mean(annual_temps[0:31])
    monthly_temps[1] = np.mean(annual_temps[31:59])
    monthly_temps[2] = np.mean(annual_temps[59:90])
    monthly_temps[3] = np.mean(annual_temps[90:120])
    monthly_temps[4] = np.mean(annual_temps[120:151])
    monthly_temps[5] = np.mean(annual_temps[151:181])
    monthly_temps[6] = np.mean(annual_temps[181:212])
    monthly_temps[7] = np.mean(annual_temps[212:243])
    monthly_temps[8] = np.mean(annual_temps[243:273])
    monthly_temps[9] = np.mean(annual_temps[273:304])
    monthly_temps[10] = np.mean(annual_temps[304:334])
    monthly_temps[11] = np.mean(annual_temps[334:365])
    return np.round(monthly_temps, 2)

def calculate_monthly_precip_from_daily(annual_precip):
    monthly_precip = np.zeros(12)
    monthly_precip[0] = np.sum(annual_precip[0:31])
    monthly_precip[1] = np.sum(annual_precip[31:59])
    monthly_precip[2] = np.sum(annual_precip[59:90])
    monthly_precip[3] = np.sum(annual_precip[90:120])
    monthly_precip[4] = np.sum(annual_precip[120:151])
    monthly_precip[5] = np.sum(annual_precip[151:181])
    monthly_precip[6] = np.sum(annual_precip[181:212])
    monthly_precip[7] = np.sum(annual_precip[212:243])
    monthly_precip[8] = np.sum(annual_precip[243:273])
    monthly_precip[9] = np.sum(annual_precip[273:304])
    monthly_precip[10] = np.sum(annual_precip[304:334])
    monthly_precip[11] = np.sum(annual_precip[334:365])
    return monthly_precip

def assign_initial_daily_precip(monthly_precip):
    precip_daily = np.zeros(365)
    precip_daily[0:31] = monthly_precip[0]/31
    precip_daily[31:59] = monthly_precip[1]/28
    precip_daily[59:90] = monthly_precip[2]/31
    precip_daily[90:120] = monthly_precip[3]/30
    precip_daily[120:151] = monthly_precip[4]/31
    precip_daily[151:181] = monthly_precip[5]/30
    precip_daily[181:212] = monthly_precip[6]/31
    precip_daily[212:243] = monthly_precip[7]/31
    precip_daily[243:273] = monthly_precip[8]/30
    precip_daily[273:304] = monthly_precip[9]/31
    precip_daily[304:334] = monthly_precip[10]/30
    precip_daily[334:365] = monthly_precip[11]/31
    return precip_daily

def compute_daily_temps_runAvg(temps_daily, temps_mth, cycle_val):
    temps_temp = np.zeros(730)
    temps_temp[0:365] = temps_daily[0:365]
    temps_temp[365:730] = temps_daily[0:365]
    for i in range(0, cycle_val):
        for j in range(0, 730-4):
            temps_temp[j+2] = np.mean(temps_temp[j:j+4])
        temps_daily_new_runAvg = np.zeros(365)
        temps_daily_new_runAvg[0:10] = temps_temp[365:10+365]
        temps_daily_new_runAvg[-10:] = temps_temp[365-10:365]
        temps_daily_new_runAvg[10:365-10] = (
            np.array(temps_temp[10:365-10]) + np.array(temps_temp[365+10:730-10])
        )/2.0
        temps_daily_new_mths_rA = calculate_mean_monthly_temps(temps_daily_new_runAvg)
        temps_diff_ = temps_daily_new_mths_rA - temps_mth
        if i < cycle_val:
            temps_daily_new_runAvg[0:31] -= temps_diff_[0]
            temps_daily_new_runAvg[31:59] -= temps_diff_[1]
            temps_daily_new_runAvg[59:90] -= temps_diff_[2]
            temps_daily_new_runAvg[90:120] -= temps_diff_[3]
            temps_daily_new_runAvg[120:151] -= temps_diff_[4]
            temps_daily_new_runAvg[151:181] -= temps_diff_[5]
            temps_daily_new_runAvg[181:212] -= temps_diff_[6]
            temps_daily_new_runAvg[212:243] -= temps_diff_[7]
            temps_daily_new_runAvg[243:273] -= temps_diff_[8]
            temps_daily_new_runAvg[273:304] -= temps_diff_[9]

```

```

        temps_daily_new_runAvg[304:334] -= temps_diff_[10]
        temps_daily_new_runAvg[334:365] -= temps_diff_[11]
        temps_temp[0:365] = temps_daily_new_runAvg[0:365]
        temps_temp[365:730] = temps_daily_new_runAvg[0:365]
    temps_temp_ = np.zeros(730)
    for k in range(0+10, 730-10):
        temps_temp_[k] = np.mean(temps_temp[k-10:k+11])
    temps_temp_[0:10] = temps_temp[365:365+10]
    temps_temp_[730-10:] = temps_temp[365-10:365]
    temps_temp = temps_temp_
    return temps_temp[:365] # return 365 values (the notebook returns 730; see note below)

def calculate_wetbulb(drybulb, dp_temps):
    c1 = 17.625
    c2 = 243.04
    vals1 = c1*dp_temps[0:365] / (dp_temps[0:365] + c2)
    vals2 = c1*drybulb[0:365] / (drybulb[0:365] + c2)
    rel_humid = 100*np.exp(vals1)/np.exp(vals2)
    c3 = np.power(rel_humid, 1.5)
    c4 = np.power(rel_humid+8.313659, 0.5)
    c5 = np.arctan(0.151977*c4)
    c6 = np.arctan(0.023101*rel_humid)
    c7 = np.arctan(rel_humid-1.676331)
    c8 = np.arctan(drybulb+rel_humid)
    wetbulb = drybulb*c5 + 0.00391838*c3*c6 - c7 + c8 - 4.686035
    return wetbulb

def calculate_dp_from_RH(drybulb, rel_humid):
    # source: https://en.wikipedia.org/wiki/Dew_point#Calculating_the_dew_point
    dp = np.zeros(12)
    c1 = 17.625
    c2 = 243.04
    vals1 = np.log(rel_humid/100)
    vals2 = np.divide(c1*drybulb[0:12], (drybulb[0:12] + c2))
    dp = np.divide(c2*(vals1 + vals2), (c1 - (vals1 + vals2)))
    return dp

def subtract_offsets(curr_data_daily, offset_data_mth):
    curr_data_daily[0:31] -= offset_data_mth[0]/31
    curr_data_daily[31:59] -= offset_data_mth[1]/28
    curr_data_daily[59:90] -= offset_data_mth[2]/31
    curr_data_daily[90:120] -= offset_data_mth[3]/30
    curr_data_daily[120:151] -= offset_data_mth[4]/31
    curr_data_daily[151:181] -= offset_data_mth[5]/30
    curr_data_daily[181:212] -= offset_data_mth[6]/31
    curr_data_daily[212:243] -= offset_data_mth[7]/31
    curr_data_daily[243:273] -= offset_data_mth[8]/30
    curr_data_daily[273:304] -= offset_data_mth[9]/31
    curr_data_daily[304:334] -= offset_data_mth[10]/30
    curr_data_daily[334:365] -= offset_data_mth[11]/31
    return curr_data_daily

def smoothing_31dayavg(input_array, sumboolean):
    input_base = np.zeros(730)
    input_base[0:365] = input_array
    input_base[365:730] = input_array
    input_smooth = np.zeros(730)
    for i in range(0+15, 730-15):
        if sumboolean:
            input_smooth[i] = np.sum(input_base[i-15:i+16])
        else:
            input_smooth[i] = np.mean(input_base[i-15:i+16])
    input_smooth[0:15] = input_smooth[365:365+15]
    input_smooth[730-15:] = input_smooth[365-15:365]
    return input_smooth[0:365]

def Daylight(latitude, day):
    P = math.asin(0.39795 * math.cos(0.2163108 + 2 * math.atan(0.9671396 *
        math.tan(.00860 * (day - 186))))))
    pi = math.pi
    daylightamount = 24 - (24 / pi) * math.acos(
        (math.sin((0.8333 * pi / 180) + math.sin(latitude * pi / 180) * math.sin(P)) /

```

```

        (math.cos(latitude * pi / 180) * math.cos(P))))
    return daylightamount

def total_possible_sunlight_totals(latitude):
    hours = np.zeros(365)
    monthly_sum = np.zeros(12)
    for i in range(365):
        hours[i] = Daylight(latitude, i)
    for i in range(0, 31):
        monthly_sum[0] += hours[i]
        monthly_sum[2] += hours[i+59]
        monthly_sum[4] += hours[i+120]
        monthly_sum[6] += hours[i+181]
        monthly_sum[7] += hours[i+212]
        monthly_sum[9] += hours[i+273]
        monthly_sum[11] += hours[i+334]
    for i in range(0, 30):
        monthly_sum[3] += hours[i+90]
        monthly_sum[5] += hours[i+151]
        monthly_sum[8] += hours[i+243]
        monthly_sum[10] += hours[i+304]
    for i in range(0, 28):
        monthly_sum[1] += hours[i+31]
    return monthly_sum

def total_possible_sunlight_totals_daily(latitude):
    hours = np.zeros(365)
    for i in range(365):
        hours[i] = Daylight(latitude, i)
    return hours

```

```

# =====
# SYLHET INPUT DATA (from the Wikipedia climate box)
# https://en.wikipedia.org/wiki/Sylhet#Geography\_and\_climate
# =====
climate_name = "Sylhet_Bangladesh_1991-2020"
latitude = 24 + 54/60      # 24.9 degrees N

# Monthly temperature means (deg C)
highs_mth = np.array([25.6, 28.2, 31.2, 31.5, 31.4, 31.6,
                      32.2, 32.5, 32.3, 31.8, 29.7, 26.8])
lows_mth = np.array([ 9.8, 12.4, 17.4, 21.2, 23.0, 24.8,
                      25.3, 25.3, 24.8, 22.3, 16.6, 11.8])
avgs_mth = np.array([18.4, 20.8, 24.3, 26.0, 26.8, 27.6,
                      28.0, 28.2, 27.9, 26.7, 23.3, 19.7])

# Relative humidity (%)
rel_humid_mth = np.array([75, 68, 68, 76, 81, 87, 87, 86, 86, 83, 77, 75])
# We start with dewpoint all zeros so the code takes the RH branch
dewpoint_mth = np.zeros(12)

# Precipitation (mm)
precip_mth = np.array([7, 38, 127, 382, 590, 795,
                       723, 609, 496, 201, 25, 10])

# compute precip_d_thresh for graphing precipitation
precip_d_thresh = np.ceil((max(precip_mth)/28)/5)*5

# Precipitation days (>= 1 mm)
precip_days_mth = np.array([1, 3, 7, 15, 22, 26, 28, 26, 20, 10, 2, 1])

# Record extremes (deg C)
rec_high_mth = np.array([34.5, 35.0, 38.8, 40.5, 38.2, 39.6,
                         38.4, 37.9, 38.3, 37.2, 35.3, 31.3])
rec_low_mth = np.array([ 3.4,  8.8, 11.5, 14.0, 18.0, 18.0,
                         19.3, 22.3, 20.5, 16.5, 13.4,  8.0])

# Sunshine hours (Wikipedia gives hours, notebook wants percent)
sun_hrs = np.array([212.2, 210.6, 223.2, 196.1, 178.6, 121.9,
                    132.8, 145.1, 148.7, 218.9, 242.9, 238.0])

np.set_printoptions(suppress=True)
time = np.linspace(0, 730, 730)

```

```

# =====
# DEWPOINT + SUNSHINE SETUP
# =====
# Notebook takes the "RH branch" when dewpoint_mth is all zeros
# need to handle case where relative humidity stats are for PM

dewpoint_mth0 = np.zeros(12)

if not hasDewpoint and (np.all(dewpoint_mth == 0)) and not (np.all(rel_humid_mth == 0)):
    # RH given; compute dewpoint from mean daily high temps
    dewpoint_mth = np.round(calculate_dp_from_RH(highs_mth, rel_humid_mth), 4)
    print("Dewpoint from highs_mth: ")
    print(dewpoint_mth)
    if not humidityPM:
        # RH given and not for PM; compute dewpoint from mean daily average temps
        dewpoint_mth0 = np.round(calculate_dp_from_RH(avgs_mth, rel_humid_mth), 4)
        print("Dewpoint from avgs_mth: ")
        print(dewpoint_mth0)
elif hasDewpoint and not (np.all(dewpoint_mth == 0)):
    print("Dewpoints: ")
    print(dewpoint_mth)
elif not hasDewpoint and (np.all(rel_humid_mth == 0)):
    print("Error: please double check your humidity values.")

# Maximum possible daylight hours for the latitude
max_sun_hrs = total_possible_sunlight_totals(latitude)
print("Possible sunshine hours for " + climate_name)
print(max_sun_hrs)

if not sunshinePerc:
    # Convert Sylhet sunshine HOURS into the PERCENT array the notebook expects
    sun_percent = np.round(sun_hrs / max_sun_hrs * 100, 2)

sun_percent_alt = np.average(sun_percent)
print("Monthly sunshine percent:")
print(sun_percent)
print("Average sunshine percent:", sun_percent_alt)

# =====
# DAILY TEMPERATURE + DEWPOINT
# =====
high_temps = assign_monthly_avg_to_days(highs_mth)
low_temps = assign_monthly_avg_to_days( lows_mth)
avg_temps = assign_monthly_avg_to_days(avgs_mth)
RH_temps = assign_monthly_avg_to_days(rec_high_mth)
RL_temps = assign_monthly_avg_to_days(rec_low_mth)

# if daily average humidity, use dew points calculated from average temps
if not humidityPM:
    dewpoint_temps = assign_monthly_avg_to_days(dewpoint_mth0)
    dp_temps_new = compute_daily_temps_runAvg(dewpoint_temps, dewpoint_mth0, 50)
# if afternoon average humidity, use dew points calculated from high temps
elif humidityPM:
    dewpoint_temps = assign_monthly_avg_to_days(dewpoint_mth)
    dp_temps_new = compute_daily_temps_runAvg(dewpoint_temps, dewpoint_mth, 50)

high_temps_new_ = compute_daily_temps_runAvg(high_temps, highs_mth, 50)
highs_mth_compute365 = calculate_mean_monthly_temps(high_temps_new_)
print("Highs: daily reconstruction minus source monthly means:")
print(highs_mth_compute365 - highs_mth)

low_temps_new_ = compute_daily_temps_runAvg(low_temps, lows_mth, 50)
lows_mth_compute365 = calculate_mean_monthly_temps(low_temps_new_)
print("Lows: daily reconstruction minus source monthly means:")
print(lows_mth_compute365 - lows_mth)

high_temps_new_ = 1.0*np.round(1*high_temps_new_, 2)
low_temps_new_ = 1.0*np.round(1*low_temps_new_, 2)
avg_temps_new_ = (np.array(low_temps_new_) + np.array(high_temps_new_))/2.0
dp_temps_new_ = 1.0*np.round(1*dp_temps_new, 2)

wetbulb_high_dp = calculate_wetbulb(high_temps_new_[0:365], dp_temps_new_)

```

```

wetbulb_low_dp = calculate_wetbulb(low_temps_new[0:365], dp_temps_new_)
wetbulb_avg_dp = calculate_wetbulb(avg_temps_new[0:365], dp_temps_new_)

sun_percent_days = assign_monthly_avg_to_days(sun_percent)
sun_percent_days_new = compute_daily_temps_runAvg(sun_percent_days, sun_percent, 50)

# =====
# PRECIPITATION: DAILY RECONSTRUCTION FROM MONTHLY TOTALS
# =====
tim = np.linspace(0, 730, 730)
p_time = np.linspace(0, 365, 365)

# Spread monthly precip totals / monthly precip-day counts evenly across days
precip_daily = assign_initial_daily_precip(precip_mth)
precip_prob_daily = assign_initial_daily_precip(precip_days_mth)

# Two-year buffers so the running average wraps smoothly
precip_temp = np.zeros(730)
precip_temp[0:365] = precip_daily[0:365]
precip_temp[365:730] = precip_daily[0:365]

precip_p_temp = np.zeros(730)
precip_p_temp[0:365] = precip_prob_daily[0:365]
precip_p_temp[365:730] = precip_prob_daily[0:365]

# Iterative smoothing: alternate between 4-day and 8-day running means
cycle_val = 10
for i in range(0, cycle_val):
    if i < cycle_val/2:
        for j in range(0, 730-4):
            precip_temp[j+2] = np.mean(precip_temp[j:j+4])
            precip_p_temp[j+2] = np.mean(precip_p_temp[j:j+4])
        else:
            for j in range(0, 730-8):
                precip_temp[j+4] = np.mean(precip_temp[j:j+8])
                precip_p_temp[j+4] = np.mean(precip_p_temp[j:j+8])

# Truncate edges of the 2-year run -> one year of daily precip
precip_daily_new_runAvg = np.zeros(365)
precip_daily_new_runAvg[0:10] = precip_temp[365:10+365]
precip_daily_new_runAvg[-10:] = precip_temp[365-10:365]
precip_daily_new_runAvg[10:365-10] = (
    np.array(precip_temp[10:365-10]) +
    np.array(precip_temp[365+10:730-10])
)/2.0

# Recompute monthly totals, then correct the daily values
precip_daily_new_mths_rA = calculate_monthly_precip_from_daily(precip_daily_new_runAvg)
precip_diff_ = precip_daily_new_mths_rA - precip_mth

# Same for the "probability of precip" array
precip_p_daily_new_runAvg = np.zeros(365)
precip_p_daily_new_runAvg[0:10] = precip_p_temp[365:10+365]
precip_p_daily_new_runAvg[-10:] = precip_p_temp[365-10:365]
precip_p_daily_new_runAvg[10:365-10] = (
    np.array(precip_p_temp[10:365-10]) +
    np.array(precip_p_temp[365+10:730-10])
)/2.0

precip_p_daily_new_mths_rA = calculate_monthly_precip_from_daily(precip_p_daily_new_runAvg)
precip_p_diff_ = precip_p_daily_new_mths_rA - precip_days_mth

if i < cycle_val:
    # Correct precip totals
    precip_daily_new_runAvg = subtract_offsets(precip_daily_new_runAvg, precip_diff_)
    precip_daily_new_runAvg[precip_daily_new_runAvg < 0] = 0
    precip_temp[0:365] = precip_daily_new_runAvg
    precip_temp[365:730] = precip_temp[0:365]

    # Correct precip probabilities (clamp to [0, 1])
    precip_p_daily_new_runAvg = subtract_offsets(precip_p_daily_new_runAvg, precip_p_diff_)
    precip_p_daily_new_runAvg[precip_p_daily_new_runAvg < 0] = 0
    precip_p_daily_new_runAvg[precip_p_daily_new_runAvg > 1] = 1
    precip_p_daily_new_runAvg[precip_p_daily_new_runAvg <= 0] = 0
    precip_p_temp[0:365] = precip_p_daily_new_runAvg
    precip_p_temp[365:730] = precip_p_temp[0:365]

```

```

# Report how well the smoothing reproduced the monthly totals
precip_D_daily_new_mths_rA = calculate_monthly_precip_from_daily(precip_daily_new_runAvg)
print("Monthly precip totals from smoothing function: \n",
      np.round(precip_daily_new_mths_rA, decimals=2))
precip_diff_ = precip_daily_new_mths_rA - precip_mth
print("Diff from source data: \n", np.round(precip_diff_, decimals=2))
print(np.sum(precip_D_daily_new_mths_rA), " versus ", np.sum(precip_mth))

precip_p_D_daily_new_mths_rA = calculate_monthly_precip_from_daily(precip_p_daily_new_runAvg)
print("Monthly precip days from smoothing function: \n",
      np.round(precip_p_daily_new_mths_rA, decimals=2))
precip_p_diff_ = precip_p_daily_new_mths_rA - precip_days_mth
print("Diff from source data: \n", np.round(precip_p_diff_, decimals=2))
print(np.sum(precip_p_D_daily_new_mths_rA), " versus ", np.sum(precip_days_mth))

# =====
# 31-DAY FLOATING WINDOW (WeatherSpark-style chart)
# =====
precip_daily_new_runAvg_sum = smoothing_31dayavg(precip_daily_new_runAvg, True)
precip_p_daily_new_runAvg_ = smoothing_31dayavg(precip_p_daily_new_runAvg, False)

running_avg_yearly_sum = calculate_monthly_precip_from_daily(precip_daily_new_runAvg_sum/31)
print("Monthly precip totals from 31-day moving average: \n",
      np.round(running_avg_yearly_sum, decimals=2))
print("Monthly precip totals (original): \n", precip_mth)
precip_diff_ = running_avg_yearly_sum - precip_mth
print("Diff from source data: \n", np.round(precip_diff_))

running_avg_yearly_sum_prob = calculate_monthly_precip_from_daily(precip_p_daily_new_runAvg_)
print("Monthly precip days count from 31-day moving average: \n",
      np.round(running_avg_yearly_sum_prob, decimals=2))
print("Monthly precip days count (original): \n", precip_days_mth)
precip_diff_p = running_avg_yearly_sum_prob - precip_days_mth
print("Diff from source data: \n", np.round(precip_diff_p, 2))

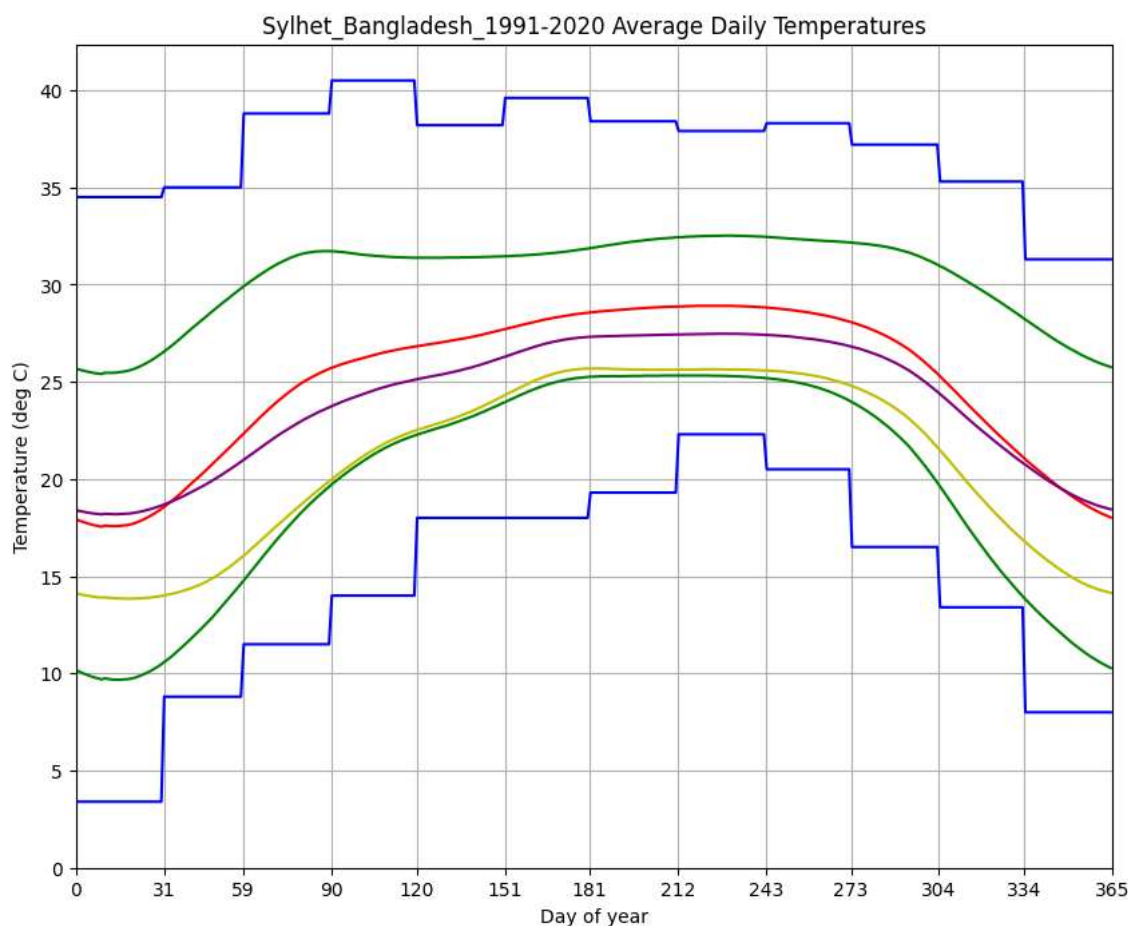
```

```

Dewpoint from highs_mth:
[20.84 21.73 24.59 26.75 27.74 29.17 29.76 29.85 29.66 28.55 25.24 22. ]
Dewpoint from avgs_mth:
[13.89 14.68 18.02 21.44 23.27 25.24 25.63 25.63 25.34 23.57 19.04 15.14]
Possible sunshine hours for Sylhet_Bangladesh_1991-2020
[333.78 315.92 371.11 381.02 412.84 409.27 419.32 403.76 370.23 360.08
 329.32 329.56]
Monthly sunshine percent:
[63.57 66.66 60.14 51.47 43.26 29.78 31.67 35.94 40.16 60.79 73.76 72.22]
Average sunshine percent: 52.451666666666675
Highs: daily reconstruction minus source monthly means:
[ 0.09 -0.01 -0.09 0.02 0. 0.02 -0.01 -0.01 0.01 -0.05 -0.01 0.04]
Lows: daily reconstruction minus source monthly means:
[ 0.1 0.04 -0.05 -0.05 0.03 -0.05 0. -0.01 -0.04 -0.09 0.05 0.05]
Monthly precip totals from smoothing function:
[ 7.34 36.57 124.52 370.98 585.19 785.55 730.1 612.26 498.06 214.05
 30.11 9.51]
Diff from source data:
[ 0.34 -1.43 -2.48 -11.02 -4.81 -9.45 7.1 3.26 2.06 13.05
 5.11 -0.49]
4004.023821284891 versus 4003
Monthly precip days from smoothing function:
[ 1.02 2.9 6.89 14.66 21.82 25.84 27.99 26.08 20.18 10.41 2.25 0.95]
Diff from source data:
[ 0.02 -0.1 -0.11 -0.34 -0.18 -0.16 -0.01 0.08 0.18 0.41 0.25 -0.05]
161.00725663513302 versus 161
Monthly precip totals from 31-day moving average:
[ 9.81 39.83 138.61 372.48 596.22 771.3 730.39 609.25 481.01 214.61
 31.81 8.68]
Monthly precip totals (original):
[ 7 38 127 382 590 795 723 609 496 201 25 10]
Diff from source data:
[ 3. 2. 12. -10. 6. -24. 7. 0. -15. 14. 7. -1.]
Monthly precip days count from 31-day moving average:
[ 1.18 3.02 7.31 14.75 21.98 25.78 27.88 25.85 19.71 10.3 2.36 0.9 ]
Monthly precip days count (original):
[ 1 3 7 15 22 26 28 26 20 10 2 1]
Diff from source data:
[ 0.18 0.02 0.31 -0.25 -0.02 -0.22 -0.12 -0.15 -0.29 0.3 0.36 -0.1 ]

```

```
# =====
# FIGURE 1: DAILY TEMPERATURES
# =====
fig, ax = plt.subplots(figsize=(10, 8))
plt.title(climate_name + " Average Daily Temperatures")
plt.xlabel("Day of year")
plt.ylabel("Temperature (deg C)")
plt.plot(time[0:365], high_temps_new[0:365], 'g')
plt.plot(time[0:365], low_temps_new[0:365], 'g')
plt.plot(time[0:365], avg_temps_new[0:365], 'r')
plt.plot(time[0:365], dp_temps_new[0:365], 'y')
plt.plot(time[0:365], wetbulb_high_dp, 'purple')
plt.plot(time[0:365], RH_temps, 'b')
plt.plot(time[0:365], RL_temps, 'b')
plt.xlim(0, 365)
ax.set_xticks([0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334, 365])
ax.set_yticks(np.arange(0, 45, 5.0)) # Sylhet: ~10-33 deg C range
ax.grid()
plt.show()
```

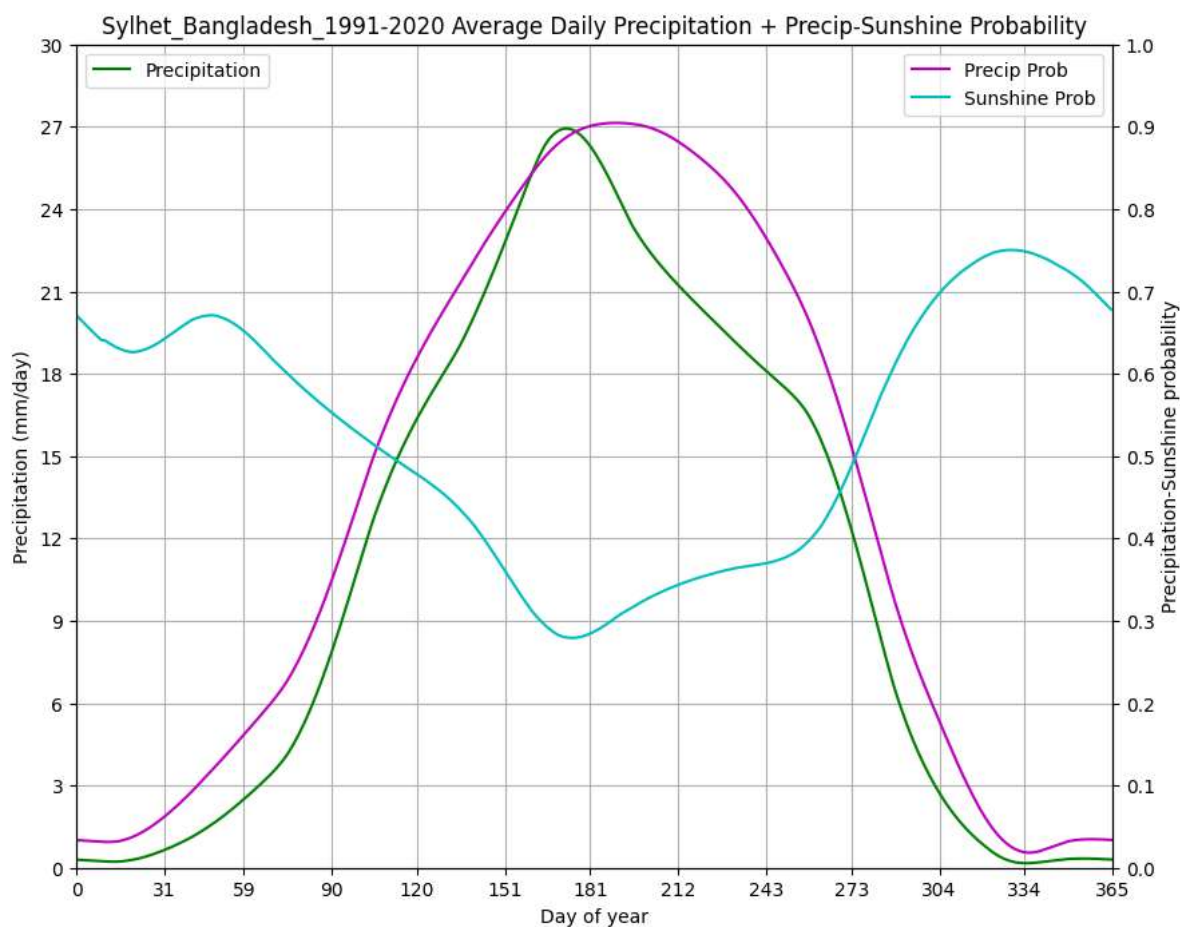


```
# =====
# FIGURE: PRECIP + PRECIP-SUNSHINE PROBABILITY
# =====
fig, ax = plt.subplots(1, figsize=(10, 8))
plt.title(climate_name + " Average Daily Precipitation + Precip-Sunshine Probability")
plt.xlabel("Day of year")
plt.ylabel("Precipitation (mm/day)")
plt.plot(tim[0:365], precip_daily_new_runAvg_sum[0:365]/31, 'g')
ax.set_xticks([0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334, 365])
ax.legend(['Precipitation'], loc='upper left')
plt.ylim(0,)
plt.xlim(0, 365)
ax2 = ax.twinx()
ax2.set_ylabel("Precipitation-Sunshine probability")
ax2.plot(tim[0:365], precip_p_daily_new_runAvg__[0:365], 'm')
ax2.plot(tim[0:365], 0.01*sun_percent_days_new[0:365], 'c')
ax2.legend(['Precip Prob', 'Sunshine Prob'], loc='upper right')
ax.grid()
```

```

ax.set_ylim(0, precip_d_thresh)
ax.set_yticks(np.linspace(0, precip_d_thresh, 11))
ax2.set_ylim(0.0, 1.0)
ax2.set_yticks(np.linspace(0, 1.0, 11))
plt.xlim(0, 365)
plt.show()

```



```

# =====
# FIGURE: PREDICTED PRECIPITATION INTENSITY
# =====
predict_precip_int = np.zeros(365)
predict_precip_int = np.where(
    precip_p_daily_new_runAvg__ > 0.01,
    np.divide(precip_daily_new_runAvg_sum[0:365]/31, precip_p_daily_new_runAvg__),
    predict_precip_int
)
predict_precip_int = np.where(
    precip_p_daily_new_runAvg__ < 0.01,
    100*precip_daily_new_runAvg_sum[0:365]/31,
    predict_precip_int
)

# Light smoothing pass on the intensity estimate
predict_precip_int_base = np.zeros(730)
predict_precip_int_base[0:365] = predict_precip_int
predict_precip_int_base[365:730] = predict_precip_int
predict_precip_int_smooth = np.zeros(730)
for i in range(0+2, 730-2):
    predict_precip_int_smooth[i] = np.mean(predict_precip_int_base[i-2:i+2])
predict_precip_int_smooth[0:2] = predict_precip_int_smooth[365:365+2]
predict_precip_int_smooth[730-2:] = predict_precip_int_smooth[365-2:365]

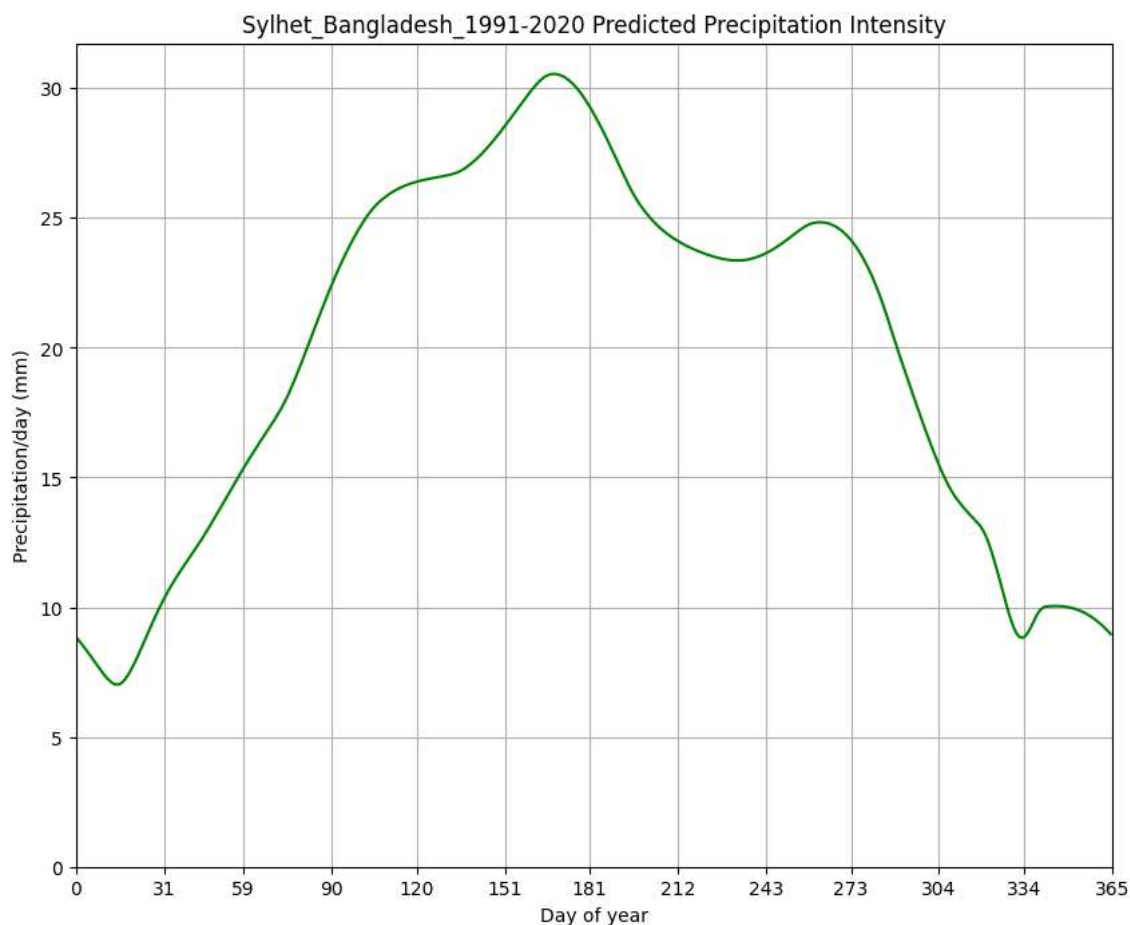
fig, ax = plt.subplots(1, figsize=(10, 8))
plt.title(climate_name + " Predicted Precipitation Intensity")
plt.xlabel("Day of year")

```

```

plt.ylabel("Precipitation/day (mm)")
plt.plot(tim[0:365], predict_precip_int_smooth[0:365], 'g')
plt.ylim(0,)
plt.xlim(0, 365)
ax.set_xticks([0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334, 365])
ax.grid()
plt.show()

```



```

# =====
# CSV EXPORT (365 x 8)
# =====
# from google.colab import files # uncomment if in Colab

avg_temps = np.zeros((365, 8))
avg_temps[:, 0] = high_temps_new_[0:365]
avg_temps[:, 1] = avg_temps_new_[0:365]
avg_temps[:, 2] = low_temps_new_[0:365]
avg_temps[:, 3] = dp_temps_new_[0:365]
avg_temps[:, 4] = 1.0*np.round(wetbulb_high_dp[0:365], 2)
avg_temps[:, 5] = 1.0*np.round(precip_daily_new_runAvg_sum[0:365], 1)
avg_temps[:, 6] = 100*np.round(precip_p_daily_new_runAvg__[0:365], 4)
avg_temps[:, 7] = np.round(sun_percent_days_new[0:365], 4)

np.set_printoptions(suppress=True, precision=2)
np.savetxt(climate_name + '_daily_averages_.csv', avg_temps, delimiter=',', fmt='%f')
print("Wrote", climate_name + "_daily_averages_.csv")

```

Wrote Sylhet_Bangladesh_1991-2020_daily_averages_.csv

